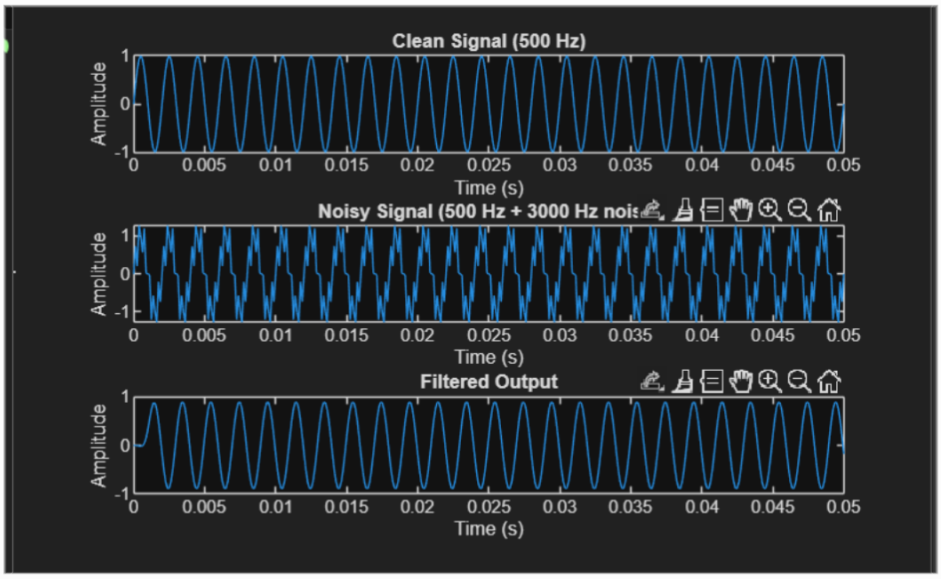
Matplot Script

```
%% PS2 - FIR Low Pass Filter Design

fs = 8000;          % Sampling frequency
fc = 1000;          % Cutoff frequency
N = 16;             % Number of taps
Wn = fc/(fs/2);     % Normalized cutoff

%% Design Filter
b = fir1(N-1, Wn, 'low', hamming(N));

%% Plot Frequency Response
figure(1);
freqz(b, 1, 1024, fs);
title('Frequency Response of 16-tap FIR Low-Pass Filter');

%% Create Noisy Signal
t = 0:1/fs:0.05;
signal_clean    = sin(2*pi*500*t);
noise           = 0.5*sin(2*pi*3000*t);
signal_noisy    = signal_clean + noise;

%% Apply Filter
signal_filtered = filter(b, 1, signal_noisy);

%% Plot All Three Signals
figure(2);
subplot(3,1,1);
plot(t, signal_clean);
title('Clean Signal (500 Hz)');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(3,1,2);
plot(t, signal_noisy);
title('Noisy Signal (500 Hz + 3000 Hz noise)');
xlabel('Time (s)'); ylabel('Amplitude');

subplot(3,1,3);
plot(t, signal_filtered);
title('Filtered Output');
xlabel('Time (s)'); ylabel('Amplitude');

%% Print Coefficients
```

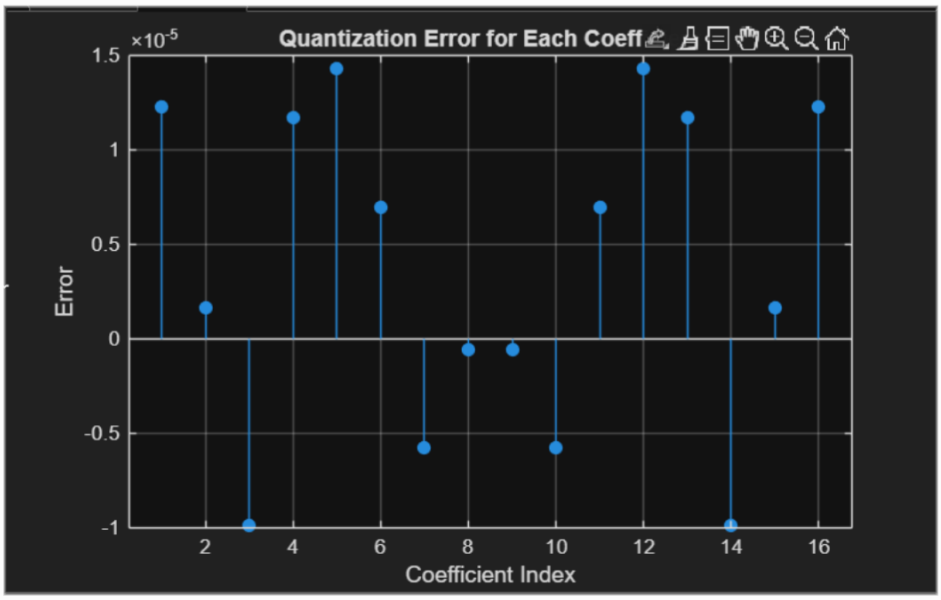
```
disp('Filter Coefficients:');
disp(b);
```

Clear Signal, Noisy Signal, and Filtered Signal



Filter coefficients:

-0.0013	-0.0054	-0.0124	-0.0107	0.0204	0.0904	0.1784	0.2406
0.2406	0.1784	0.0904	0.0204	-0.0107	-0.0124	-0.0054	-0.0013



FIXED POINT CONVERSION

```
%% Step 2: Fixed Point Conversion (Q1.15 format)

b = [-0.0013 -0.0054 -0.0124 -0.0107 0.0204 0.0904 ...
     0.1784 0.2406 0.2406 0.1784 0.0904 0.0204 ...
    -0.0107 -0.0124 -0.0054 -0.0013];

% Convert to Q1.15
scale = 2^15;                    % = 32768
b_fixed = round(b * scale);         % multiply and round

% Calculate quantization error
b_reconstructed = b_fixed / scale;   % convert back to decimal
quant_error = b - b_reconstructed;   % difference = error

% Display results
disp('Original | Q1.15 Integer | Reconstructed | Error');
for i = 1:length(b)
   fprintf('%.4f   |   %6d      |   %.6f   |   %.6f\n', ...
       b(i), b_fixed(i), b_reconstructed(i), quant_error(i));
end

% Plot quantization error
figure(3);
stem(quant_error, 'filled');
title('Quantization Error for Each Coefficient');
xlabel('Coefficient Index');
ylabel('Error');
grid on;
```

CONVERSION TABLE

Original	Q1.15 Integer	Reconstructed	Error
-0.0013	-43	-0.001312	0.000012
-0.0054	-177	-0.005402	0.000002
-0.0124	-406	-0.012390	-0.000010
-0.0107	-351	-0.010712	0.000012
0.0204	668	0.020386	0.000014

0.0904	2962	0.090393	0.000007
0.1784	5846	0.178406	-0.000006
0.2406	7884	0.240601	-0.000001
0.2406	7884	0.240601	-0.000001
0.1784	5846	0.178406	-0.000006
0.0904	2962	0.090393	0.000007
0.0204	668	0.020386	0.000014
-0.0107	-351	-0.010712	0.000012
-0.0124	-406	-0.012390	-0.000010
-0.0054	-177	-0.005402	0.000002
-0.0013	-43	-0.001312	0.000012

"The maximum quantization error observed is 0.000014, which is negligible and well within acceptable limits for audio signal processing applications."

TO BINARY CONVERSION FOR VERILOG USE

b[1] = -43 --> 1111111111010101
b[2] = -177 --> 1111111101001111
b[3] = -406 --> 1111111001101010

b[4] = -351 --> 1111111010100001
b[5] = 668 --> 0000001010011100
b[6] = 2962 --> 0000101110010010
b[7] = 5846 --> 0001011011010110
b[8] = 7884 --> 0001111011001100
b[9] = 7884 --> 0001111011001100
b[10] = 5846 --> 0001011011010110
b[11] = 2962 --> 0000101110010010
b[12] = 668 --> 0000001010011100
b[13] = -351 --> 1111111010100001
b[14] = -406 --> 1111111001101010
b[15] = -177 --> 1111111101001111
b[16] = -43 --> 1111111111010101

HERE IS THE RTL ARCHITECTURE DIAGRAM

```
+-----------------------------------------------+
|                FIR Filter RTL                 |
|  module fir_filter(clk, rst, x_in[15:0], y_out[31:0]) |
+-----------------------------------------------+
| Inputs: clk ──┐  rst ──┐  x_in[15:0] ──────────────────────────┤
| Outputs:      │        │                        │
| y_out[31:0] ──┘        └────────────────────────────────────────┘
+-----------------------------------------------+
                 |
                 ▼
+-----------------------------------------------+
|      16 Hardwired Coefficients (wire array)       |
| coeff[0]  = 16'hFFAD  (-43 / 16'b1111111111010101)   |
| coeff[1]  = 16'hFA4F  (-177/ 16'b1111111101001111)   |
| coeff[2]  = 16'hF9CA  (-406/ 16'b1111111001101010)   |
| coeff[3]  = 16'hFAF1  (-351/ 16'b1111111010100001)   |
| coeff[4]  = 16'h029C  ( 668/ 16'b0000001010011100)   |
| ... (symmetric, up to coeff[15] = 16'hFFAD)        |
| NO ROM - Direct assign statements                  |
+-----------------------------------------------+
                 | | |
                 ▼ ▼ ▼
+-----------------------------------------------+
|          Shift Register Array (16×16-bit)         |
|  reg signed [15:0] shift_reg[0:15]                |
|                                          |
| On posedge clk:                              |
| shift_reg[0]  <= x_in                        |
| shift_reg[1]  <= shift_reg[0]                    |
| shift_reg[2]  <= shift_reg[1]                    |
|  ...                                     |
| shift_reg[15] <= shift_reg[14]                   |
|                                          |
```

```
| On posedge rst: ALL shift_reg[i] <= 0              |
+-----------------------------------------------------+
                  |  |  |
                  ▼  ▼  ▼
+-----------------------------------------------------+
|          16 Parallel Multipliers                    |
| wire signed [31:0] products[0:15]                   |
|                                                     |
| genvar j; generate for(j=0; j<16; j++)              |
|   products[j] = shift_reg[j] * coeff[j];            |
|                                                     |
| 16×16 → 32-bit signed multiplies (combinational)    |
+-----------------------------------------------------+
                  |  |  |
                  ▼  ▼  ▼
+-----------------------------------------------------+

|          16-Input Combinational Adder               |
|                                                     |
| y_out = products[0] + products[1] + products[2] +   |
|        products[3] + products[4] + ... +            |
|        products[15];                                |
|                                                     |
| SINGLE massive assign (no pipeline registers)       |
+-----------------------------------------------------+
                  |
                  ▼
            y_out[31:0]
```

CODE FACTS FROM VERILOG

Element	Exact Code Match	RTL Block
Coefficients	`wire signed [15:0] coeff [0:15];` + 16 `assign`	Hardwired wires
Shift Reg	`reg signed [15:0] shift_reg [0:15];`	16 sequential registers
Multipliers	`generate for (j = 0; j < 16; j++)`	16 parallel combinational
Accumulator	`assign y_out = products[0] + ... + products[15];`	Single 16-input adder
Control	Only `clk`, `rst` (no enable/valid)	Minimal synchronous